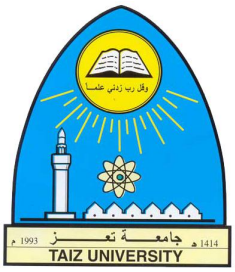

TAIZ UNIVERSITY
FACULTY OF ENGINEERING AND INFORMATION TECHNOLOGY
DEPARTMENT OF SOFTWARE ENGINEERING

TAIZ LOST AND FOUND PLATFORM

A Course Project Submitted in Partial Fulfillment of the Requirements
for the Laravel Framework Lab Course

By:
JABER MAHYOUB ALHAJ FARHAN
AMJAD ABDULHAKIM AHMED MANSOUR

Project Supervisor:
Eng. Anas Al-Sabiri

TAIZ, YEMEN
2026

Acknowledgment

We would like to express our sincere gratitude to Almighty Allah for granting us the strength, patience, and perseverance to complete this project successfully. We extend our heartfelt appreciation to our project supervisor, Dr. [Supervisor Name], whose continuous guidance, constructive feedback, and academic mentorship were invaluable throughout every phase of this work.

Abstract

The Taiz Lost & Found System is a web-based application developed to address the persistent problem of lost and found item management within Taiz Governorate, Yemen. In a region where no centralized digital infrastructure exists for reporting misplaced belongings or announcing recovered items, individuals frequently rely on inefficient social media posts or word-of-mouth communication, resulting in low recovery rates and prolonged distress for item owners.

Keywords: Lost and Found, Laravel, Web Application, Taiz, MVC Architecture, PHP, Tailwind CSS

Contents

List of Figures

List of Tables

1 Introduction

The act of losing personal belongings is a universally stressful experience that transcends cultural and geographic boundaries. From identification documents and mobile phones to house keys and wallets, the loss of everyday items can cause significant emotional distress, financial burden, and practical inconvenience. In well-connected urban environments, centralized lost-and-found services operated by transportation authorities, shopping malls, and municipal offices often facilitate the recovery process. However, in many developing regions, including Taiz Governorate in Yemen, such institutional support is either absent or severely limited.

Taiz, the third-largest city in Yemen by population, presents unique challenges for lost item recovery. The city's complex urban geography — spanning numerous neighborhoods, districts, and administrative divisions — combined with limited public infrastructure and the disruptions caused by years of conflict, has created an environment where recovering lost belongings is exceedingly difficult. Residents typically resort to posting on fragmented social media groups, relying on personal networks, or simply accepting the loss.

The Taiz Lost Found System was conceived as a direct response to this community need. The platform aims to serve as a centralized, accessible, and user-friendly digital hub where any resident of Taiz can report a lost item or announce a found item, browse existing listings, and establish direct contact with the relevant party. By localizing the system to Taiz — with a comprehensive database of over 130 neighborhoods and districts covering all major administrative divisions — the platform provides granular geographic context that dramatically increases the probability of successful item recovery.

From a technical perspective, the project leverages the Laravel PHP framework, one of the most widely adopted web application frameworks in the world, known for its elegant syntax, robust security features, and comprehensive ecosystem. The choice of Laravel enabled rapid development without sacrificing code quality, security, or maintainability. The front-end was built using Tailwind CSS, a utility-first CSS framework that facilitated the creation of a modern, responsive, and visually appealing Arabic (RTL) interface.

This report documents the complete lifecycle of the Taiz Lost Found System — from problem identification and requirements analysis through system design, implementation, testing, and deployment. It is structured to provide both a comprehensive technical reference and a reflective account of the development process, including the challenges encountered and lessons learned.

2 Problem Statement

In Taiz Governorate, the absence of a dedicated, centralized system for managing lost and found items creates a cascading set of problems that affect residents on a daily basis. The

following specific issues have been identified through observation and community engagement:

2.1 Fragmentation of Information Channels

Currently, individuals who lose or find items in Taiz resort to posting on various social media platforms, primarily Facebook groups and WhatsApp broadcasts. These channels are inherently fragmented — a lost item posted in one group may never be seen by the person who found it in a different group. There is no single, authoritative platform where all lost-and-found reports are aggregated.

2.2 Lack of Geographic Precision

Social media posts rarely provide structured geographic information. A vague description such as ”lost near the market” is insufficient in a city with dozens of markets and commercial areas. The absence of standardized neighborhood categorization makes it extremely difficult to correlate lost items with found items based on location proximity.

2.3 Information Overload and Poor Searchability

Social media feeds are chronological and ephemeral. A lost item post made three days ago is effectively buried under hundreds of newer posts, making retrospective searching impractical. There is no ability to filter by item category, type (lost vs. found), or geographic area.

2.4 Trust and Communication Barriers

Anonymous or semi-anonymous social media posts provide limited accountability. Users may hesitate to contact strangers or share personal information through public channels. A structured platform with user registration provides a baseline level of accountability and trust.

2.5 No Status Tracking or Resolution Mechanism

Once a lost item is recovered through informal channels, there is no mechanism to update or close the corresponding post. This leads to persistent ”dead” listings that waste other users’ time and attention.

2.6 Absence of Categorization and Metadata

Items are described in free-form text without standardized categories, making automated or manual matching between lost and found reports nearly impossible.

These problems collectively result in low recovery rates for lost items, wasted community effort, and ongoing frustration among Taiz residents. The Taiz Lost & Found System directly addresses each of these issues through structured data entry, geographic precision, advanced search and filtering, user authentication, and status management capabilities.

3 Project Objectives

3.1 Primary Objectives

1. **Develop a centralized web-based platform** that enables residents of Taiz Governorate to report lost items and announce found items through a unified, accessible interface.

2. **Implement a comprehensive search and filtering system** that allows users to locate relevant listings efficiently by keyword, item type (lost/found), category, and neighborhood.

3. **Provide secure user authentication and authorization** to ensure that only registered users can create listings and that only item owners can modify or delete their own announcements.

4. **Design a fully responsive, Arabic-optimized user interface** that functions seamlessly across desktop computers, tablets, and mobile devices, accommodating the RTL text direction inherent to the Arabic language.

5. **Deploy the system to a cloud hosting platform** to ensure 24/7 availability and accessibility from any internet-connected device.

3.2 Secondary Objectives

1. **Establish a localized neighborhood database** covering all major districts and administrative areas within Taiz Governorate to provide precise geographic context for each listing.

2. **Integrate direct communication channels** (phone call and WhatsApp) within item detail pages to facilitate immediate contact between item reporters and searchers.

3. **Implement an item status management system** that allows users to mark items as ”returned” once recovered, maintaining the accuracy and relevance of the platform’s listings.

4. **Optimize the platform for search engines** through dynamic sitemap generation, structured data markup, and semantic HTML to maximize the platform’s discoverability through organic search.

5. **Create a personal dashboard** where registered users can view, manage, edit, and delete all of their own listings in a centralized interface.

6. **Implement image upload functionality** to allow users to attach photographs to their listings, significantly enhancing item identifiability.

4 Importance of the Project

4.1 Social Importance

The Taiz Lost & Found System addresses a genuine community need that directly impacts residents' quality of life. By providing a centralized platform for lost and found item management, the system:

- **Reduces emotional distress** by providing a structured, hopeful pathway for recovering lost belongings, particularly important documents such as identification cards and passports.

- **Strengthens community bonds** by facilitating acts of honesty and goodwill when a finder can easily connect with an item's owner, it reinforces pro-social behavior and community trust.

- **Promotes digital inclusion** by providing a practical, locally relevant use case for technology adoption, encouraging residents who may be unfamiliar with formal web applications to engage with digital services.

- **Serves a humanitarian function** in a conflict-affected region where the loss of critical documents (identification, medical records, educational certificates) can have disproportionately severe consequences.

4.2 Organizational and Institutional Importance

- **Demonstrates local technical capacity** by showcasing that a functional, production-grade web application can be developed, deployed, and maintained by local IT professionals and students.

- **Provides a template for similar community services** that could be replicated in other Yemeni governorates or adapted for other community-oriented applications.

- **Generates practical data** about the types and frequencies of lost items, geographic hotspots, and recovery rates, which could inform future community safety and public service initiatives.

4.3 Academic Importance

- **Integrates multiple IT disciplines** web development, database design, UI/UX design, cloud deployment, and security into a cohesive, real-world application.
- **Demonstrates practical application** of the Model-View-Controller architectural pattern, a cornerstone concept in software engineering education.
- **Provides a case study** for agile development methodologies applied within resource-constrained environments.

5 Scope of the Project

5.1 System Boundaries

The Taiz Lost & Found System is scoped as a web-based application with the following defined boundaries:

In Scope:

- User registration with name, email, phone number, and password
- User authentication (login, logout, password reset)
- Profile management (update name, email, phone number; delete account)
- Item creation with fields: type (lost/found), title, description, category, neighborhood, contact phone, and optional image
- Item listing with pagination (12 items per page)
- Advanced search and filtering by keyword, type, category, and neighborhood
- Item detail view with contact information, phone link, and WhatsApp integration
- Item editing and deletion by the owning user
- Item status management (active/returned)
- User dashboard displaying all personal listings
- Responsive design supporting desktop and mobile devices
- RTL (Right-to-Left) Arabic language interface
- SEO optimization with XML sitemap and structured data
- Cloud deployment on Render with Docker containerization

Out of Scope (Current Version):

- Administrative panel for system-wide moderation
- In-app messaging or chat system between users
- Automated AI-based matching between lost and found items
- Mobile native application (iOS/Android)
- Multi-language support beyond Arabic
- Real-time push notifications
- Geographic map integration
- Reward or incentive system for finders

5.2 Target Users

The system is designed to serve two primary user categories:

User Type	Description	Access Level
Guest User	Any visitor to the platform	Browse listings, view item details, use search and filters
Registered User	A user who has created an account	All guest capabilities plus: create, edit, delete own listings; access personal dashboard; manage profile

Table 1: Target Users and Access Levels

5.3 Geographic Scope

The system is specifically tailored for Taiz Governorate, Yemen. The neighborhood database includes over 130 named areas spanning multiple administrative districts, including but not limited to:

- Mdiryat al-Qahirah (Cairo District) — central Taiz city neighborhoods
- Mdiryat al-Mudaffar
- Mdiryat Salah
- Mdiryat al-Mawasit

- Mdiryat al-Qubaitah
- Mdiryat al-Shamayiteen
- Mdiryat al-Makhadir
- Mdiryat al-Sami'i
- Mdiryat al-Misrakh
- Mdiryat al-Hajriyyah
- And additional peripheral areas

6 System Analysis

6.1 System Overview

The Taiz Lost & Found System operates as a classic client-server web application. Users interact with the system through a web browser (the client), which communicates with a Laravel-based server application via HTTP requests. The server processes requests, interacts with the database, applies business logic, and returns HTML responses rendered by the Blade templating engine.

The system follows the **Model-View-Controller (MVC)** architectural pattern:

- **Models** encapsulate the data layer and define relationships between entities (User, Item, Category, Neighborhood).
- **Views** (Blade templates) handle the presentation layer with a component-based architecture.
- **Controllers** manage the application logic, processing user input and coordinating between Models and Views.

6.2 System Actors

[PLACEHOLDER FOR SYSTEM ACTORS DIAGRAM]

Please insert the System Actors / Use Case Diagram here.

Guest User → Browse, Search, View

Registered User → CRUD Items, Dashboard, Profile

Figure 1: System Actors Diagram

Actor	Role Description
Guest User	Can browse the homepage, view all item listings, use search and filter functionality, view individual item details including contact information. Cannot create, edit, or delete listings.
Registered User	Has all guest capabilities. Additionally can: create new lost/found listings with images, edit own listings, delete own listings, mark items as returned, access personal dashboard, manage profile settings, delete account.

Table 2: System Actors and Roles

6.3 Use Cases

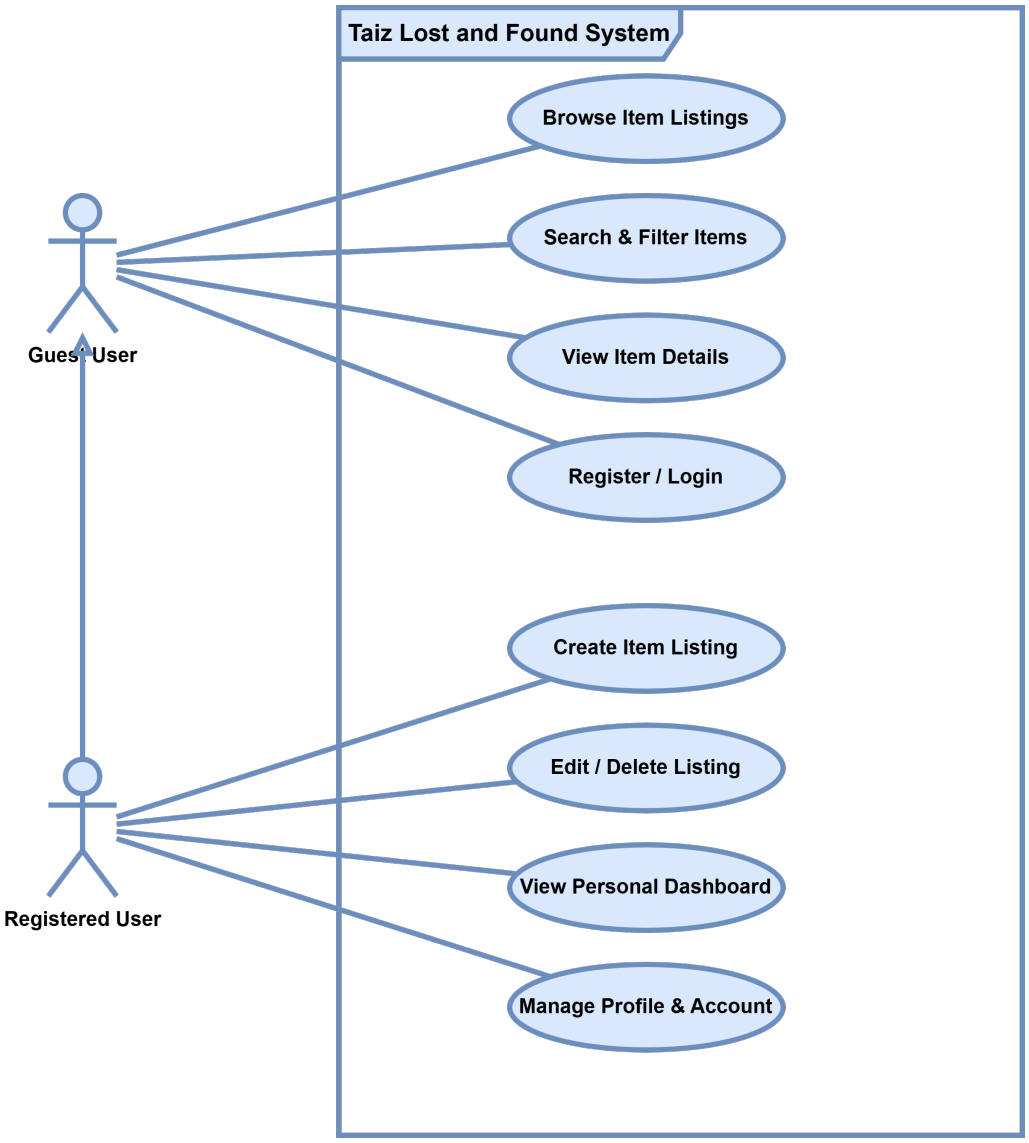

Figure 2: System Use Case Diagram showing Guest and Registered User interactions

6.4 Workflow Explanation

Item Posting Workflow

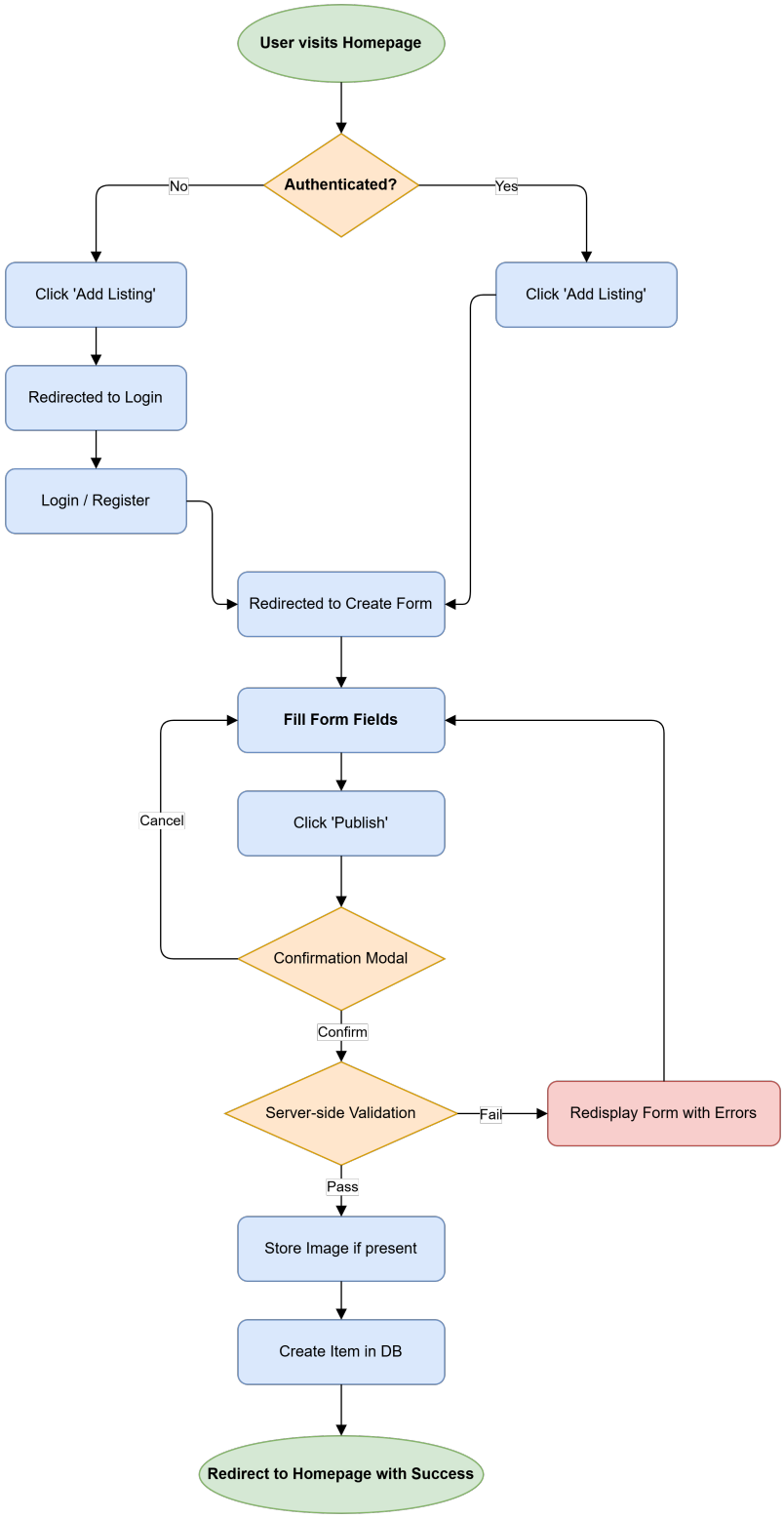

Figure 3: Item Search Workflow

Item Search Workflow

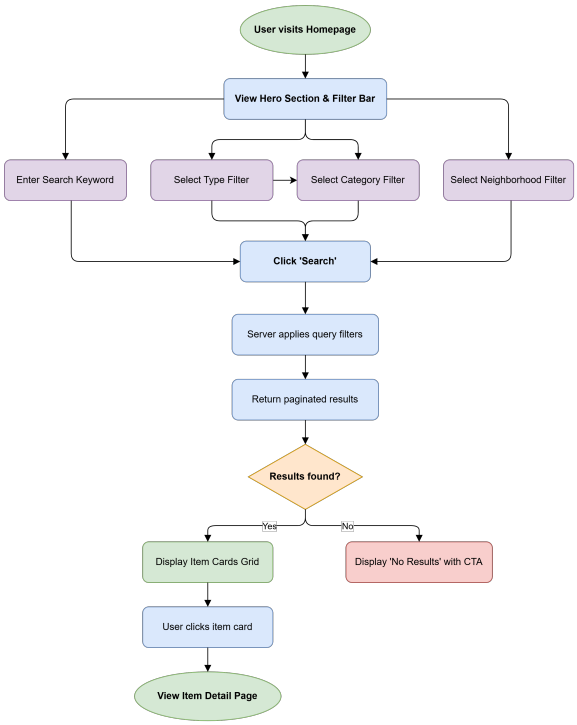

Figure 4: Item Search Workflow

6.5 Business Logic

The system implements the following key business rules:

1. **Authentication-gated operations:** Item creation, editing, and deletion require an authenticated user session. This is enforced at the controller level through the `HasMiddleware` interface, applying the `auth` middleware to all actions except `index` and `show`.

2. **Owner-based authorization:** Only the user who created an item can edit or delete it. This is enforced through controller-level `abort_if()` checks and Form Request-level authorization in `UpdateItemRequest`.

3. **Status lifecycle:** Items are created with a default status of `active`. Owners can change the status to `returned` via the edit form, indicating successful recovery.

4. **Cascade deletion:** Deleting a user cascades to all their items. Deleting a category or neighborhood also cascades to associated items.

5. **Contact auto-population:** When creating a new listing, the contact phone field is pre-populated with the user's registered phone number.

6. **Query string preservation:** Search filters are preserved across pagination through `withQueryString()`, ensuring users don't lose their filter context.

6.6 User Interaction Scenarios

Scenario 1: A resident finds a set of car keys
Ahmed, a resident of the al-Hoban neighborhood, finds a Toyota car key on the street. He opens the Taiz Lost & Found website, registers an account, and creates a new listing:

- Type: Found
- Title: Keys
- Category: Keys & Wallets
- Neighborhood: al-Hoban
- Description: Found a Toyota car key
- Contact Phone: His personal number

The listing immediately appears on the homepage. When the key owner searches for ”keys” and filters by the al-Hoban neighborhood, Ahmed’s listing appears prominently. The owner contacts Ahmed via the WhatsApp button on the item detail page.

Scenario 2: A student loses a wallet
Fatima, a university student, loses her wallet containing her student ID and some cash. She creates a ”lost” listing with a detailed description and her university neighborhood. Over the next few days, she periodically checks the platform by filtering for ”found” items in the ”Keys & Wallets” category. When a matching found item appears, she contacts the finder. After recovering her wallet, she edits her listing and changes the status to ”Returned”.

7 Functional Requirements

The following table enumerates the system’s functional requirements with unique identifiers, detailed descriptions, and priority levels.

Table 3: Functional Requirements

Req. ID	Requirement Description	Priority
FR-01	The system shall allow guest users to browse all item listings on the homepage in a paginated grid layout (12 items per page).	High
FR-02	The system shall allow users to search for items by keyword, matching against both the title and description fields using partial string matching.	High

FR-03	The system shall allow users to filter listings by item type (lost or found).	High
FR-04	The system shall allow users to filter listings by item category from a predefined set of categories.	High
FR-05	The system shall allow users to filter listings by neighborhood from a comprehensive list of Taiz neighborhoods.	High
FR-06	The system shall allow multiple filters to be applied simultaneously and preserve filter selections across pagination.	High
FR-07	The system shall display individual item detail pages showing: title, type badge, status badge, description, category, neighborhood, poster name, contact phone, posting date, and image (if available).	High
FR-08	The system shall provide a direct phone call link and a WhatsApp deep link on item detail pages for contacting the item reporter.	High
FR-09	The system shall allow new users to register with: full name, email address, phone number, and password (with confirmation).	High
FR-10	The system shall enforce unique constraints on email addresses and phone numbers during registration.	High
FR-11	The system shall allow registered users to log in using email and password credentials.	High
FR-12	The system shall provide a ”Remember Me” option for persistent login sessions.	Medium
FR-13	The system shall allow registered users to request a password reset via email.	Medium
FR-14	The system shall allow authenticated users to create new item listings with fields: type (lost/found), title, description, category, neighborhood, contact phone, and optional image upload.	High
FR-15	The system shall validate all form inputs server-side: required fields, string length limits, enum value constraints, foreign key existence checks, and image file type/size validation (max 5MB).	High
FR-16	The system shall automatically associate new item listings with the authenticated user’s account.	High

FR-17	The system shall set the default status of new items to ”active.”	High
FR-18	The system shall allow item owners to edit all fields of their own listings, including changing the status from ”active” to ”returned.”	High
FR-19	The system shall allow item owners to delete their own listings permanently, with a confirmation dialog.	High
FR-20	The system shall prevent non-owners from editing or deleting listings (HTTP 403 Forbidden).	High
FR-21	The system shall provide a personal dashboard page displaying all items posted by the authenticated user, with edit and delete actions.	High
FR-22	The system shall allow authenticated users to update their profile information (name, email, phone number).	Medium
FR-23	The system shall allow authenticated users to delete their account permanently, requiring password confirmation.	Medium
FR-24	The system shall invalidate email verification status when a user changes their email address.	Medium
FR-25	The system shall display success/error flash messages with animated toast notifications after key actions (item creation, update, deletion).	Medium
FR-26	The system shall display custom confirmation modals before destructive or irreversible actions (deletion, publishing).	Medium
FR-27	The system shall generate a dynamic XML sitemap listing the homepage and all individual item pages.	Low
FR-28	The system shall visually distinguish returned items from active items (grayscale images, ”Returned” badge, reduced opacity).	Medium
FR-29	The system shall pre-populate the contact phone field with the user’s registered phone number when creating a new listing.	Low
FR-30	The system shall support image uploads in PNG, JPG, and GIF formats, storing them on the public disk.	High

8 Non-Functional Requirements

8.1 Security

Table 4: Security Requirements

ID	Requirement	Implementation
NFR-S1	All passwords must be stored as irreversible hashes	Bcrypt hashing via Laravel's `hashed` cast on the User model
NFR-S2	All state-changing forms must include CSRF protection	`@csrf` Blade directive on every form; `VerifyCsrfToken` middleware
NFR-S3	User input must be validated server-side before processing	Dedicated `StoreItemRequest` and `UpdateItemRequest` Form Request classes
NFR-S4	Authorization must prevent cross-user data manipulation	`abort_if()` checks and FormRequest `authorize()` methods
NFR-S5	SQL injection must be prevented	Eloquent ORM parameterized queries; no raw SQL
NFR-S6	Session data must be invalidated on logout and account deletion	`session()->invalidate()` and `session()->regenerateToken()` in destroy methods
NFR-S7	Password reset tokens must be stored securely	Dedicated `password_reset_tokens` table with hashed tokens
NFR-S8	The `password` and `remember_token` fields must be hidden from serialization	`#[Hidden]` attribute on User model

8.2 Performance

Table 5: Performance Requirements

ID	Requirement	Implementation
NFR-P1	Database queries must be optimized to prevent N+1 problems	Eager loading with `with(['user', 'category', 'neighborhood'])`
NFR-P2	The sitemap must be cached to reduce database load	30-minute cache via `Cache::remember()`
NFR-P3	Assets must be compiled and minified for production	Vite build process (`npm run build`)
NFR-P4	Pagination must limit the number of records per page	12 items per page via `paginate(12)`
NFR-P5	Images must be size-limited to prevent storage abuse	Maximum 5MB enforced via validation rule `max:5120`

8.3 Scalability

- The application architecture cleanly separates concerns (MVC), allowing individual layers to be modified or replaced independently.
- The database schema uses proper foreign key indexing, supporting efficient joins as data grows.
- The Dockerized deployment model facilitates horizontal scaling if needed.
- The seed-based approach for categories and neighborhoods allows easy data expansion without schema changes.

8.4 Reliability

- The application uses database transactions implicitly through Eloquent for atomic operations.
- Foreign key constraints with `cascadeOnDelete()` ensure referential integrity.
- The deployment includes a health check endpoint (`/debug-health`) for monitoring database connectivity and view rendering.
- Error handling returns appropriate HTTP status codes (403 for unauthorized, 404 for not found).

8.5 Usability

- The interface is fully RTL-optimized for Arabic language users.
- The Cairo Google Font is used throughout for optimal Arabic text rendering.
- All forms preserve old input on validation failure (`old()` helper).
- Interactive confirmation modals prevent accidental destructive actions.
- Visual feedback is provided through animated toast notifications.
- The responsive design adapts to mobile screens via Tailwind CSS breakpoints.
- Navigation includes a hamburger menu for mobile devices.

8.6 Maintainability

- The codebase follows PSR-4 autoloading standards.
- Controllers use dedicated Form Request classes for validation logic separation.
- The Blade templating engine uses a component-based architecture (`x-app-layout`, `x-guest-layout`, etc.).
- Database structure is version-controlled through migration files.
- Seed data is programmatically defined for reproducible environments.
- Composer scripts provide a unified development command.

9 Database Design and Analysis

9.1 Overview

The Taiz Lost & Found System uses a relational database consisting of six primary tables and three supporting tables. The database is managed through Laravel's migration system, ensuring version-controlled schema evolution and reproducible deployments.

9.2 Main Database Tables

9.2.1 `users` Table

Table 6: Users Table Schema

Column	Type	Description & Constraints
`id`	BIGINT (unsigned)	PRIMARY KEY, AUTO_INCREMENT
`name`	VARCHAR(255)	NOT NULL, User's full name
`email`	VARCHAR(255)	NOT NULL, UNIQUE
`phone_number`	VARCHAR(255)	NOT NULL, UNIQUE
`password`	VARCHAR(255)	NOT NULL, Bcrypt-hashed password
`timestamps`	TIMESTAMP	`created_at`, `updated_at`, `email_verified_at`

9.2.2 `items` Table

Table 7: Items Table Schema

Column	Type	Description & Constraints
`id`	BIGINT (unsigned)	PRIMARY KEY, AUTO_INCREMENT
`title`	VARCHAR(255)	NOT NULL, Item title/name
`description`	TEXT	NOT NULL, Detailed item description
`image_path`	VARCHAR(255)	NULLABLE, Path to uploaded image
`type`	ENUM	NOT NULL, ('lost', 'found')
`status`	ENUM	NOT NULL, DEFAULT 'active' ('active', 'returned')
`contact_phone`	VARCHAR(255)	NOT NULL, Contact phone number
`user_id`	BIGINT (unsigned)	FOREIGN KEY → users(id), CASCADE DELETE
`category_id`	BIGINT (unsigned)	FOREIGN KEY → categories(id), CASCADE DELETE
`neighborhood_id`	BIGINT (unsigned)	FOREIGN KEY → neighborhoods(id), CASCADE DELETE
`timestamps`	TIMESTAMP	`created_at`, `updated_at`

9.2.3 Categories and Neighborhoods Tables

Table 8: Lookup and Supporting Tables

Table	Details
`categories`	`id` (PK), `name` (VARCHAR). Seeded with categories such as Electronics, Documents, Keys, Bags, Jewelry, etc.
`neighborhoods`	`id` (PK), `name` (VARCHAR). Seeded with over 130 neighborhoods covering major districts of Taiz.
`Supporting Tables`	`password_reset_tokens`, `sessions`, `cache`, `jobs` for system background processing.

9.3 Entity Relationships

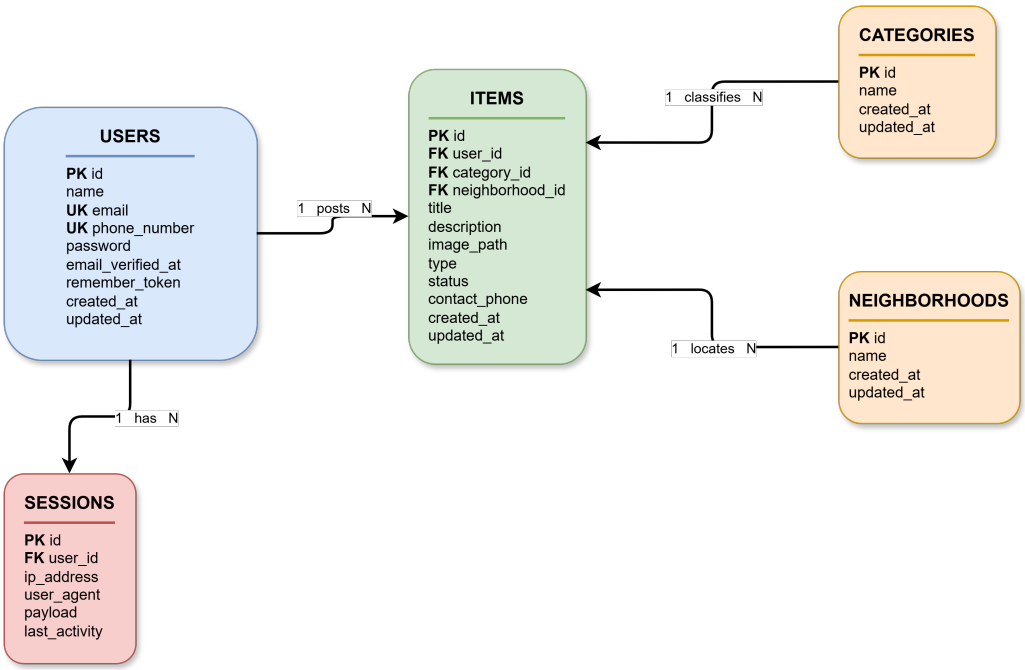

Figure 5: Item Search Workflow

9.4 Relationship Summary

Table 9: Database Relationships Summary

Relationship	Type	Description	Cascade Rule
User → Items	One-to-Many	A user can post multiple items	ON DELETE CASCADE
Category → Items	One-to-Many	A category can classify multiple items	ON DELETE CASCADE
Neighborhood → Items	One-to-Many	A neighborhood can locate multiple items	ON DELETE CASCADE
User → Sessions	One-to-Many	A user can have multiple active sessions	—

10 System Design

10.1 Frontend Structure

The system's frontend is built using Laravel's Blade templating engine with a component-based architecture. The view hierarchy is organized as follows:

```
resources/views/
   layouts/
       app.blade.php          # Main authenticated layout
       guest.blade.php        # Guest/auth pages layout
       navigation.blade.php   # Shared navigation bar
   components/
       app-layout.blade.php
       confirm-modal.blade.php
       flash-message.blade.php
       dropdown.blade.php
       nav-link.blade.php
       ... (15 component files)
   items/
       index.blade.php        # Homepage with hero + listings
       create.blade.php       # New item form
       edit.blade.php         # Edit item form
       show.blade.php         # Item detail page
```

```
auth/
    login.blade.php
    register.blade.php
    forgot-password.blade.php
    reset-password.blade.php
    verify-email.blade.php
    confirm-password.blade.php
profile/
    edit.blade.php
    partials/
dashboard.blade.php        # User's personal listings
welcome.blade.php          # Default Laravel welcome (unused)
```

10.2 Navigation Design

The navigation system is implemented as a sticky, semi-transparent top navigation bar with backdrop blur effect. It adapts between desktop and mobile layouts:

Desktop Navigation:

- Logo (left/right depending on RTL) with hover animation
- Primary links: Homepage (الرئيسية), My Listings (إعلاناتي) (authenticated only)
- Quick action button: "Add Listing" (إضافة إعلان) (authenticated only)
- User dropdown: Profile (حسابي), Logout (تسجيل الخروج)
- Guest actions: Login (تسجيل الدخول), Register (حساب جديد)

Mobile Navigation:

- Hamburger menu toggle with animated icon transition
- Full-width responsive links
- User info section with name and email display

10.3 Responsive Design

The system implements a mobile-first responsive design using Tailwind CSS breakpoint utilities:

Table 10: Responsive Breakpoints

Breakpoint	**Width**	**Layout Behavior**
Default (mobile)	$<$ 640px	Single column grid, hamburger navigation, full-width cards
`sm`	$\geq$ 640px	Rounded corners on cards, side-by-side form fields begin
`md`	$\geq$ 768px	2-column grid for listings, side-by-side dashboard cards
`lg`	$\geq$ 1024px	3-column grid for listings, "Add Listing" button in nav

10.4 Forms and Validation

The system includes four primary forms, each with comprehensive validation:

Item Creation Form:

- Type selection via styled radio card buttons (lost/found)
- Text input for title (max 255 characters)
- Dropdown selects for category and neighborhood (populated from database)
- Textarea for description
- Phone input with RTL support (pre-populated from user profile)
- File upload with drag-and-drop zone styling (PNG/JPG/GIF, max 5MB)
- Confirmation modal before submission

Item Edit Form:

- All fields from creation form, pre-populated with existing values
- Additional status field (active/returned) via radio cards
- Current image preview with option to replace
- PUT method spoofing via `@method('PUT')`

Registration Form:

- Full name, phone number (with LTR input direction), email, password, password confirmation

- All fields validated server-side with appropriate rules

Login Form:

- Email and password fields
- ”Remember Me” checkbox
- ”Forgot Password” link

All forms implement:

- `@csrf` token protection
- Server-side validation via Form Request classes
- `old()` value preservation on validation failure
- `x-input-error` component for inline error display
- Visual feedback through focus ring animations and background color transitions

10.5 User Dashboard

The dashboard (accessible at `/dashboard`, requires authentication) presents:

- **Header:** ”My Listings” (إعلاناتي الخاصة) with total listing count
- **Quick Action:** ”Add New Listing” button with gradient styling
- **Empty State:** Illustrated empty state with descriptive text when no listings exist
- **Listing Cards:** Two-column grid (on medium screens and above) showing:
  - Item thumbnail (with grayscale filter for returned items)
  - Item title (linked to detail page)
  - Type badge (lost/found)
  - Status badge (returned, if applicable)
  - Creation date
  - Hover-reveal action buttons (Edit, Delete)
- **Delete Confirmation:** Custom modal with danger styling for permanent deletion

10.6 Color System and Design Tokens

The application uses a custom color palette defined in the Tailwind configuration:

Table 11: Design Tokens and Color Palette

Token	Hex Value	Usage
`primary`	`#0F7A5C`	Primary brand green — buttons, links, accents
`primary-light`	`#18A07A`	Lighter green — gradient endpoints, hover states
`primary-dark`	`#0B5C45`	Darker green — hover states, hero backgrounds
`brand-text`	`#2F2F2F`	Primary text color
`brand-bg`	`#F9FAFB`	Page background, input backgrounds

Typography: Cairo Google Font (Arabic-optimized, weights 400–700)

11 Technologies Used

The following table summarizes the core technologies and tools utilized throughout the development, testing, and deployment phases of the system.

Table 12: Technology Stack

Technology	Version	Role in the System
PHP	8.3	Server-side programming language.
Laravel	13.0	Full-stack PHP framework (MVC, ORM, Routing).
Laravel Breeze	2.4	Authentication scaffolding.
MySQL / SQLite	—	Relational database management.
Tailwind CSS	4.x	Utility-first CSS framework for frontend styling.
Blade	Built-in	Template engine for views.
Vite	—	Frontend build and asset compilation tool.
Alpine.js	—	Lightweight JavaScript for interactive UI components.
Docker	—	Containerization platform for consistent deployment.
Nginx	—	Web server and reverse proxy.
Render	—	Cloud hosting and CI/CD platform.
Composer	2.x	PHP dependency management.
Git	—	Version control system.
Cairo Font	—	Arabic-optimized web font family.

12 Why Laravel Was Chosen

Laravel was selected as the primary framework for this project based on a comprehensive evaluation of its architectural features, developer productivity advantages, security capabilities, and ecosystem maturity. The following subsections detail the specific framework features that directly benefited this project.

12.1 MVC Architecture

Laravel enforces the Model-View-Controller (MVC) architectural pattern, which provides clear separation of concerns. This was particularly valuable in this project, allowing parallel development of the database schema and the user interface.

12.2 Routing System

Laravel's expressive routing system enabled clean URL design, significantly reducing boilerplate code:

[bgcolor=gray!10, frame=lines]php // RESTful routes defined in one line Route::resource('items', ItemController::class); Route::get('/', [ItemController::class, 'index'])->name('home');

12.3 Eloquent ORM

Laravel's Eloquent ORM was instrumental in managing the relational data model. It simplified complex queries through fluent API chains, as demonstrated in our search and filter implementation:

[bgcolor=gray!10, frame=lines]php $query->when$(request->filled('search'), function (q)use(request) $q->where('title','like',')$;

12.4 Authentication Scaffolding

Laravel Breeze provided a complete, production-ready authentication system out of the box, including registration, login, password reset, and email verification. This saved an estimated 15–20 hours of development time compared to building authentication from scratch.

12.5 Security Features

Laravel's built-in security features addressed multiple threat vectors without requiring additional third-party libraries:

Table 13: Laravel Security Features

Threat	**Laravel Protection**
SQL Injection	Eloquent ORM uses parameterized queries exclusively
CSRF Attacks	Automatic token verification on all POST/PUT/DELETE routes
XSS Attacks	Blade's `{{ }}` syntax auto-escapes output by default
Password Theft	`hashed` cast auto-hashes passwords via Bcrypt
Session Hijacking	Session regeneration on login and logout

12.6 Blade Templating Engine

Blade's component-based architecture enabled modular, reusable UI composition. Using layouts (`x-app-layout`) and reusable UI components (`x-confirm-modal`, `x-flash-message`) ensured consistent design and reduced code duplication across the application.

12.7 Security Features

Laravel's built-in security features addressed multiple threat vectors without requiring additional third-party libraries:

Table 14: Security Features and Protections

Threat	**Laravel Protection**
SQL Injection	Eloquent ORM uses parameterized queries exclusively
Cross-Site Request Forgery (CSRF)	Automatic token verification on all POST/PUT/DELETE routes
Cross-Site Scripting (XSS)	Blade's `{{ }}` syntax auto-escapes output by default
Password Theft	`hashed` cast auto-hashes passwords via Bcrypt
Session Hijacking	Session regeneration on login and logout
Mass Assignment	`$fillable` arrays on all models whitelist allowed attributes

13 Authentication and Security

13.1 Login System

The authentication system is implemented through Laravel Breeze's `AuthenticatedSessionController`. The login flow proceeds as follows:

1. **Submission:** The user submits their email and password via the login form.
2. **Validation:** The `LoginRequest` validates the input format and enforces rate limiting to prevent brute-force attacks.
3. **Verification:** Laravel's `Auth::attempt()` method verifies the provided password against the hashed version stored in the database.

4. **Session Creation:** Upon successful verification, a new session is created, and the user is redirected to their intended destination (typically the dashboard).

5. **Error Handling:** On failure, the system returns a throttled error response, ensuring that attackers cannot easily guess or brute-force valid credentials.

The login process is protected against common vulnerabilities using Laravel's built-in session management and authentication guards.

[bgcolor=gray!10, frame=lines]php // Example of the authentication attempt logic public function store(LoginRequest $request) : $RedirectResponse$request->authenticate(); // Throttles and validates credentials $request->session()->regenerate(); //Preventssessionfixation
return redirect()->intended(route('dashboard', absolute: false));

13.2 Password Hashing

Passwords are never stored in plain text. The `User` model automatically processes the `password` attribute through the Bcrypt algorithm using Laravel's modern model casting feature:

[bgcolor=gray!10, frame=lines]php protected function casts(): array return ['password' => 'hashed', // Auto-hashes using Bcrypt (cost 12)];

13.3 CSRF Protection

Every form in the application is fortified with a Cross-Site Request Forgery (CSRF) token via the `@csrf` Blade directive. This generates a hidden input field containing a cryptographically random token that is validated by Laravel's `VerifyCsrfToken` middleware on all POST, PUT, PATCH, and DELETE requests.

[bgcolor=gray!10, frame=lines]html <form action="/items" method="POST"> @csrf </form>

This mechanism ensures that any request submitted to the server originated from our legitimate application, effectively preventing malicious third-party sites from executing unauthorized actions on behalf of authenticated users.

13.4 Input Validation

All user input is validated server-side through dedicated Form Request classes. This approach centralizes validation logic, keeping controllers clean and ensuring data integrity before processing.

Example: `StoreItemRequest` validation rules [bgcolor=gray!10, frame=lines]php public function rules(): array return ['type' => 'required|in:lost,found', 'title' => 'required|string|max:255', 'description' => 'required|string', 'category$_i d'$ =>' $required|exists : categories, id'$,' $neighborhood_i d'$ =

$required|exists:neighborhoods,id','contact_phone' => 'required|string|max:20','image' => 'nullable|image|max:5120', //5MB limit];$

Key validation benefits include:

- **Type Safety:** Enforces input formats and values (e.g., `enum` constraints).
- **Referential Integrity:** The `exists` rule verifies that foreign keys (Category/Neighborhood IDs) actually exist in the database.
- **Data Sanitization:** String limits prevent oversized inputs, and image validation ensures only valid file formats are accepted.
- **Automatic Feedback:** If validation fails, Laravel automatically redirects the user back with comprehensive error messages and preserves the user's previously entered input using the `old()` helper.

13.5 Session Handling

Laravel manages user sessions using a secure, driver-based approach. By default, the application utilizes the `file` session driver, which stores session data in an encrypted format on the server.

Key session security practices implemented in this system:

- **Session Regeneration:** The system triggers `session()->regenerate()` upon successful authentication to protect against session fixation attacks.
- **Secure Storage:** Session cookies are marked as `httpOnly` and `secure` (if HTTPS is enabled), preventing access by client-side JavaScript.
- **Timeout and Invalidation:** Sessions are explicitly invalidated during logout or account deletion, ensuring that no stale session identifiers remain valid.
- **CSRF Tokens:** The session driver maintains CSRF tokens, which are verified on every state-changing request.

13.6 Authorization

Authorization ensures that users can only interact with resources they own. This is strictly enforced through a multi-layered approach:

1. Controller-Level Authorization: Using Laravel's `authorize()` method within controllers, we prevent unauthorized access to specific actions: [bgcolor=gray!10, frame=lines]php

public function edit(Item $item)//$Throws 403 Forbidden if the user is not the owner$this->authorize('update $item); $return view('items.edit', compact('item'));$

2. Policy-Based Access Control: We use a dedicated `ItemPolicy` to centralize authorization logic: [bgcolor=gray!10, frame=lines]php public function update(User $user, $Item$item): bool // Users can only update their own listings return $user- > id ===$item->user$_id$;

3. Form Request Authorization: For critical actions like updating or deleting items, the Form Request classes themselves contain an `authorize()` method that performs the check before validation even begins, providing an extra layer of defense-in-depth.

14 Project Implementation Process

The project followed an agile-inspired iterative development cycle, divided into distinct phases to ensure a structured approach from conception to production.

14.1 Planning Phase

The project began with a comprehensive planning phase that included:

- **Needs Assessment:** Informal interviews with Taiz residents regarding lost item recovery experiences and current community practices.
- **Market Analysis:** Review of existing lost-and-found platforms to identify best practices and usability gaps.
- **Technology Selection:** Evaluation of PHP frameworks (Laravel vs. alternatives) and frontend strategies.
- **Scope Definition:** Establishing clear in-scope and out-of-scope boundaries.

14.2 Analysis Phase

During analysis, the team:

- Identified primary user roles (Guest and Registered User).
- Documented 14 use cases with detailed main and alternative flows.
- Defined 30 functional requirements and six quality dimensions for non-functional requirements.
- Formulated the Entity-Relationship Diagram (ERD).

14.3 Design Phase

The design phase produced:

- **Database schema:** Primary tables (users, items, categories, neighborhoods) and supporting tables.
- **UI wireframes:** Blueprints for the homepage, detail views, forms, dashboard, and auth pages.
- **Design system:** Color palette (#0F7A5C primary green) and Cairo typography.
- **Component architecture:** Blueprint for modular Blade components.

14.4 Development Phase

Development proceeded iteratively across four primary sprints:

Table 15: Development Sprints Overview

Sprint	Focus Area
Sprint 1	Initialization (Breeze, DB migrations, Eloquent models, Seeding).
Sprint 2	Core Logic (ItemController CRUD, Form Requests, Search/Filter).
Sprint 3	UI/UX (Tailwind styling, RTL layout, Cairo font, Responsive nav).
Sprint 4	Deployment (Docker, Nginx, Render configuration, SEO).

14.5 Testing Phase

Testing was conducted at multiple levels to ensure reliability and security:

- **Automated Testing:** Unit and feature testing for authentication and CRUD operations.
- **Cross-Platform Testing:** Manual validation across browsers (Chrome, Firefox, Safari) and device breakpoints.
- **Security Testing:** Authorization boundary validation to prevent cross-user data manipulation.
- **Health Monitoring:** Debugging endpoints for production state checks.

14.6 Deployment Phase

The application was deployed via a containerized cloud pipeline:

1. **CI/CD:** Multi-stage Docker build (Composer, Vite, Nginx).
2. **Configuration:** Environment secrets managed via Render dashboard.
3. **Provisioning:** Database migration/seed execution on deployment.
4. **Infrastructure:** Automated SSL via Render with production URL: `https://taiz-lost-and-foun` `onrender.com`.

15 Challenges Faced During Development

This section outlines the primary technical and environmental challenges encountered during the project implementation and the solutions devised to overcome them.

15.1 Challenge 1: RTL Layout Complexity

Problem: Implementing a fully Right-to-Left (RTL) interface required reversing numerous CSS assumptions. Tailwind CSS utility classes like padding and margin (`pl-`, `pr-`) required careful adjustment, and phone number inputs required `dir="ltr"` overrides to maintain correct digit ordering within an RTL context.
Solution: Used the `dir="rtl"` attribute on the `<html>` element, systematically tested each component, and applied `dir="ltr"` specifically to LTR-content inputs (email, phone). Tailwind's RTL support handled the majority of directional properties automatically.

15.2 Challenge 2: Image Storage on Ephemeral Hosting

Problem: Render's environment utilizes ephemeral storage, where the filesystem is reset on every deployment, risking the loss of user-uploaded images.
Solution: While local storage was used for development, the `Storage::url()` helper was implemented consistently to abstract the storage location, facilitating a future transition to persistent storage solutions like AWS S3 or Cloudinary.

15.3 Challenge 3: Database Configuration for Cloud Deployment

Problem: The application needed to be flexible enough to support SQLite (for development) and PostgreSQL/MySQL (for production), requiring specific PHP extensions in the Docker environment.
Solution: The Docker image was configured to include `pdo_sqlite`, `pdo_mysql`, and `pdo_pgsql` PHP extensions. The database driver is dynamically managed via environment variables.

15.4 Challenge 4: 500 Errors on Initial Deployment

Problem: The initial deployment encountered 500 Internal Server errors with limited visibility into server logs.
Solution: Created a custom `/debug-health` route that independently checks database connectivity, model accessibility, and view rendering, providing immediate JSON-based diagnostic feedback to identify root causes like missing Vite manifests.

15.5 Challenge 5: Neighborhood Data Management

Problem: Taiz features a complex administrative structure with no publicly available, definitive digital registry of neighborhoods.
Solution: Compiled a custom registry of over 130 neighborhoods through cross-referencing government records and community input. Implemented `firstOrCreate()` within the seeder to handle data integrity during initialization.

15.6 Challenge 6: Arabic Text Search Accuracy

Problem: Arabic morphological complexity (e.g., diacritics, pluralization) posed challenges for standard substring matching.
Solution: Implemented broad `LIKE` matching with dual-sided wildcard operators. While this satisfies core requirements, full morphological stemming has been flagged as a priority for future enhancements.

15.7 Challenge 7: Custom Confirmation Modals

Problem: The project required custom-styled, reusable confirmation modals without the overhead of heavy JavaScript frameworks.
Solution: Developed a reusable Blade component (`x-confirm-modal`) powered by Alpine.js. This component supports dynamic titles, action-specific text, and color variants (danger/warning), triggered via a lightweight `askConfirm()` JavaScript helper function.

16 Future Enhancements

To ensure the long-term sustainability and impact of the Taiz Lost & Found System, several high-value features have been identified for future implementation phases.

16.1 AI and Computer Vision Integration

- **AI-Powered Item Matching:** Implementation of machine learning models to analyze item attributes (category, keywords, location) and suggest high-probability matches

using NLP optimized for Arabic.

- **OCR for Documents:** Integration of Optical Character Recognition to extract text from images of lost documents (IDs, passports), enabling indexing and searchable document details.
- **Visual Search:** Utilizing Computer Vision APIs (e.g., Google Vision) to enable ”search by photo” and automated item categorization.

16.2 Platform and Communication Expansion

- **Mobile Application:** Developing native iOS/Android apps using Flutter or React Native to support GPS-based reporting, push notifications, and offline access.
- **Real-Time Notifications:** Deployment of a WebSocket-based system (using Laravel Reverb) for instant alerts on item status changes or potential matches.
- **In-App Chat:** A secure messaging system allowing users to coordinate item recovery without sharing private contact information.

16.3 Geographic and Administrative Enhancements

- **Maps Integration:** Utilizing Google Maps or OpenStreetMap for precise location pinning, interactive map views, and proximity-based searching.
- **Admin Dashboard:** A centralized panel for user management, item moderation, system health monitoring, and advanced analytics (e.g., recovery rate heatmaps).

16.4 User Experience and Global Reach

- **Multi-Language Support:** Extending the UI to support English and other languages using Laravel localization.
- **Social Media Integration:** Leveraging Laravel Socialite for OAuth login (Google, Facebook) and auto-sharing features to increase listing visibility in community groups.

17 Conclusion

The Taiz Lost & Found System represents a meaningful intersection of academic learning and community service. Through the development of this platform, we have demonstrated that modern web technologies — specifically the Laravel PHP framework, Tailwind CSS, and cloud hosting infrastructure — can be effectively leveraged to address real-world challenges facing communities in developing regions.

The system successfully delivers on its core objectives: providing a centralized, searchable, and user-friendly platform for lost and found item management within Taiz Governorate. With its comprehensive neighborhood database, multi-criteria search capability, secure authentication system, and responsive Arabic interface, the platform offers a significantly superior alternative to the fragmented social media channels currently used by Taiz residents.

From a technical standpoint, the project reinforced several fundamental software engineering principles:

1. **Separation of Concerns:** The MVC architecture proved invaluable in managing complexity, allowing independent development and modification of data, logic, and presentation layers.

2. **Security by Design:** Laravel's built-in security features (CSRF protection, password hashing, parameterized queries, mass assignment protection) demonstrated the importance of choosing frameworks that prioritize security as a default behavior rather than an afterthought.

3. **Iterative Development:** The sprint-based approach allowed early identification of deployment challenges and progressive refinement of the user interface based on testing feedback.

4. **Cloud Deployment Practicality:** Containerizing the application with Docker and deploying to Render demonstrated the accessibility of modern cloud platforms for student and community projects.

While the current version of the system represents a functional and deployable Minimum Viable Product (MVP), the extensive list of future enhancements — from AI-powered matching to mobile applications and real-time notifications — illustrates the rich potential for continued development. The architectural decisions made during this initial phase (clean MVC separation, RESTful routing, component-based frontend) provide a solid foundation for these future extensions.

We are hopeful that this project, beyond fulfilling its academic requirements, will serve as a practical tool for the residents of Taiz and a template for similar community-oriented technology initiatives across Yemen and the broader region.

References

[1] Otwell, T. (2026). *Laravel Framework Documentation (Version 13.x)*. Retrieved from https://laravel.com/docs

[2] Stauffer, M. (2024). *Laravel: Up and Running* (3rd ed.). O'Reilly Media.

[3] Tailwind Labs. (2025). *Tailwind CSS Documentation*. Retrieved from https://tailwindcss.com/docs

[4] Welling, L., & Thomson, L. (2024). *PHP and MySQL Web Development* (6th ed.). Addison-Wesley Professional.

[5] Lockhart, J. (2015). *Modern PHP: New Features and Good Practices*. O'Reilly Media.

[6] Pressman, R. S., & Maxim, B. R. (2024). *Software Engineering: A Practitioner's Approach* (10th ed.). McGraw-Hill Education.

[7] Sommerville, I. (2023). *Software Engineering* (11th ed.). Pearson Education.

[8] Elmasri, R., & Navathe, S. B. (2022). *Fundamentals of Database Systems* (7th ed.). Pearson Education.

[9] Duckett, J. (2022). *HTML and CSS: Design and Build Websites*. John Wiley & Sons.

[10] Mozilla Developer Network. (2026). *Web Technology for Developers: HTTP, HTML, CSS, JavaScript*. Retrieved from https://developer.mozilla.org

[11] Render. (2026). *Render Documentation: Web Services*. Retrieved from https://render.com/docs

[12] Docker Inc. (2026). *Docker Documentation*. Retrieved from https://docs.docker.com

[13] PHP Group. (2026). *PHP 8.3 Documentation*. Retrieved from https://www.php.net/docs.php

[14] OWASP Foundation. (2024). *OWASP Top Ten Web Application Security Risks*. Retrieved from https://owasp.org/www-project-top-ten/

[15] International Organization for Standardization. (2023). *ISO/IEC 25010:2023 — Systems and Software Quality Models*. ISO.

[16] Nielsen, J. (2020). *10 Usability Heuristics for User Interface Design*. Nielsen Norman Group. Retrieved from https://www.nngroup.com/articles/ten-usability-heuristics/

[17] Al-Wabil, A., & Al-Khalifa, H. S. (2021). Web accessibility for Arabic content: Challenges and best practices. *Journal of King Saud University – Computer and Information Sciences*, 33(5), 612–623.

[18] Google Fonts. (2026). *Cairo Font Family*. Retrieved from https://fonts.google.com/specimen/Cairo